

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – Shorting



NAMA: Syahrul adi wibowo

NIM: 109082500222

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Sorting atau pengurutan data merupakan proses menyusun sekumpulan data ke dalam urutan tertentu agar data lebih mudah dikelola dan diproses. Urutan tersebut dapat berupa ascending (menaik) dari nilai terkecil ke terbesar maupun descending (menurun) dari nilai terbesar ke terkecil. Dalam pemrograman, sorting sangat penting karena banyak proses lain seperti searching, filtering, dan pengolahan laporan akan menjadi lebih efisien apabila data telah tersusun dengan rapi.

Pada praktikum ini digunakan dua algoritma sorting dasar, yaitu Selection Sort dan Insertion Sort. Kedua algoritma tersebut termasuk algoritma sederhana yang sering digunakan untuk memahami konsep dasar pengurutan sebelum mempelajari algoritma yang lebih kompleks seperti Merge Sort, Heap Sort, atau Quick Sort. Walaupun sama-sama memiliki kompleksitas waktu $O(n^2)$, cara kerja kedua algoritma tersebut berbeda.

Selection Sort bekerja dengan cara memilih elemen minimum atau maksimum dari bagian array yang belum terurut, kemudian menempatkannya pada posisi yang sesuai. Proses pencarian dilakukan menggunakan perulangan bersarang.

Perulangan pertama menentukan posisi yang akan diisi, sedangkan perulangan kedua mencari elemen terkecil atau terbesar pada bagian data yang tersisa. Setelah ditemukan, elemen tersebut ditukar menggunakan proses swap. Algoritma ini memiliki kelebihan berupa jumlah pertukaran data yang sedikit dan implementasi yang mudah dipahami.

Insertion Sort bekerja dengan metode penyisipan data. Algoritma ini menganggap sebagian data di awal array sudah terurut, kemudian mengambil satu elemen dari bagian yang belum terurut untuk dimasukkan ke posisi yang tepat. Selama proses penyisipan, elemen-elemen lain akan digeser agar tersedia ruang kosong. Cara kerja algoritma ini menyerupai proses mengurutkan kartu secara manual. Insertion Sort cukup efisien untuk data yang jumlahnya kecil atau data yang hampir terurut karena pada kondisi tertentu dapat bekerja lebih cepat dibanding algoritma sederhana lainnya.

Selain diterapkan pada array integer, algoritma sorting juga dapat digunakan pada tipe data lain seperti string dan struct. Pada tipe string, pengurutan dilakukan berdasarkan urutan alfabet atau nilai Unicode dari setiap karakter. Sedangkan pada tipe struct, proses pengurutan dilakukan menggunakan field tertentu sebagai acuan, misalnya IPK mahasiswa atau rating buku. Penggunaan struct memungkinkan beberapa data yang saling berkaitan disimpan dalam satu variabel sehingga pengelolaan data menjadi lebih terstruktur.

Praktikum ini juga memanfaatkan konsep array statis menggunakan `const nMax` sebagai batas maksimum data. Tipe array didefinisikan menggunakan keyword `type` agar program lebih mudah dibaca dan digunakan kembali. Selain itu, parameter function dikirim menggunakan pointer sehingga perubahan data di dalam function langsung memengaruhi array asli tanpa perlu melakukan penyalinan data. Teknik ini membuat penggunaan memori menjadi lebih efisien, terutama ketika jumlah data cukup besar.

Pada beberapa kasus, data yang telah terurut digunakan kembali untuk proses pencarian menggunakan Binary Search. Algoritma Binary Search bekerja dengan membandingkan data yang dicari terhadap elemen tengah array, kemudian mempersempit area pencarian menjadi setengah bagian secara berulang. Karena area pencarian terus dibagi dua, algoritma ini memiliki kompleksitas $O(\log n)$ sehingga jauh lebih cepat dibanding pencarian linear.

Melalui praktikum sorting ini, dapat dipahami bahwa pemilihan algoritma sangat dipengaruhi oleh kondisi data dan kebutuhan program. Pemahaman mengenai sorting menjadi dasar penting dalam pengembangan program yang berkaitan dengan pengolahan data dalam jumlah besar.

B. Guided

1. Guided 1: Array dan Slice

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
/* I.S. terdefinisi array T yang berisi n bilangan bulat
F.S. array T terurut secara mengecil atau descending dengan INSERTION SORT
*/
```

```
var temp, i, j int

i = 1
for i <= n-1 {
    j = i
    temp = T[j]

    for j > 0 && temp > T[j-1] {
        T[j] = T[j-1]
        j = j - 1
    }

    T[j] = temp
    i = i + 1
}

func main() {
    var data arrInt
    var n int

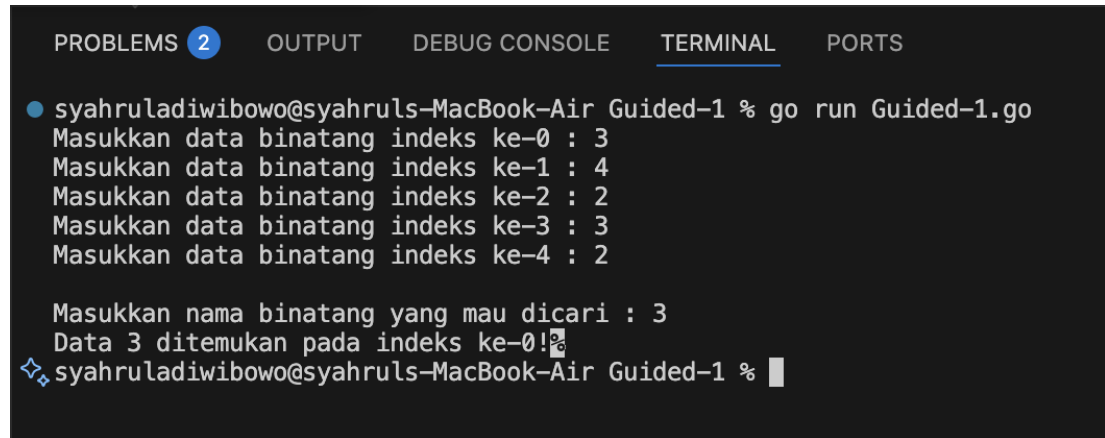
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}
```

Output :



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● syahruladiwibowo@syahruls-MacBook-Air Guided-1 % go run Guided-1.go
Masukkan data binatang indeks ke-0 : 3
Masukkan data binatang indeks ke-1 : 4
Masukkan data binatang indeks ke-2 : 2
Masukkan data binatang indeks ke-3 : 3
Masukkan data binatang indeks ke-4 : 2

Masukkan nama binatang yang mau dicari : 3
Data 3 ditemukan pada indeks ke-0!
❖ syahruladiwibowo@syahruls-MacBook-Air Guided-1 %
```

Penjelasan:

Program tersebut digunakan untuk mengurutkan data integer menggunakan algoritma Selection Sort secara ascending atau dari nilai terkecil ke terbesar. Data disimpan pada array berukuran tetap yang kemudian diisi melalui input pengguna. Pada function `selecetionSortArray`, proses sorting dilakukan dengan dua buah perulangan. Perulangan luar berfungsi menentukan posisi elemen yang akan diisi, sedangkan perulangan dalam digunakan untuk mencari nilai paling kecil dari sisa data yang belum terurut. Indeks dari nilai terkecil tersebut disimpan pada variabel `idx_min`.

Apabila ditemukan data yang lebih kecil dibanding elemen sebelumnya, maka indeks minimum akan diperbarui. Setelah seluruh proses pencarian selesai, dilakukan pertukaran data menggunakan teknik swap agar elemen terkecil berpindah ke posisi yang benar.

Pada function `main`, program meminta pengguna memasukkan lima buah angka, kemudian menampilkan data sebelum dan sesudah proses sorting dilakukan. Setelah function sorting dipanggil, array yang awalnya tidak terurut akan berubah menjadi urut menaik.

2. Guided 2

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int

    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i

        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK {
                idx_min = j
            }
        }

        if idx_min != i {
            sMHS[i], sMHS[idx_min] =
            sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)

        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
```

```
fmt.Print("NIM : ")
fmt.Scan(&structMHS[i].nim)

fmt.Print("IPK : ")
fmt.Scan(&structMHS[i].IPK)
}

fmt.Println()
fmt.Println("=== SEBELUM SORTING ===")

for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}

fmt.Println("=====")
fmt.Println()

SelectionSortArray(&structMHS)

fmt.Println("=== SETELAH SORTING ===")

for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}

fmt.Println("=====")
}
```

Output :

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL POP
● syahruladiwibowo@syahruls-MacBook-Air Guided-2 % go run
--- Masukkan Data Mahasiswa Ke-0 ---
Nama : adit
NIM : 12021
IPK : 2323
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : sjao
NIM : 21
IPK : wsq
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : NIM : 31
IPK : sas
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : NIM : wqw
IPK : 3122
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : wq
NIM : 23w
IPK : qww

=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : adit
NIM : 12021
IPK : 2323.00
--- Data Mahasiswa Ke-1 ---
Nama : sjao
NIM : 21
IPK : 0.00
--- Data Mahasiswa Ke-2 ---
Nama : sq
NIM : 31
```

Penjelasan:

Program tersebut menerapkan algoritma Selection Sort pada array bertipe struct mahasiswa. Struct digunakan untuk menyimpan beberapa atribut mahasiswa seperti nama, NIM, dan IPK dalam satu kesatuan data.

Pengurutan dilakukan berdasarkan nilai IPK secara ascending. Pada function SelectionSortArray, program mencari mahasiswa dengan nilai IPK paling kecil

pada bagian array yang belum terurut. Posisi data tersebut disimpan menggunakan variabel `idx_min`.

Perbandingan dilakukan melalui field IPK pada struct mahasiswa. Jika ditemukan mahasiswa dengan IPK yang lebih kecil, maka indeks minimum diperbarui. Setelah proses pencarian selesai, seluruh isi struct akan ditukar sehingga data nama, NIM, dan IPK tetap sesuai dengan pemiliknya.

Pada bagian main, pengguna diminta memasukkan data lima mahasiswa.

Program kemudian menampilkan data sebelum sorting dan sesudah sorting.

Setelah proses pengurutan selesai, data mahasiswa akan tersusun berdasarkan IPK dari yang terkecil ke terbesar.

.

3. Guided 3

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int

    i = 1

    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

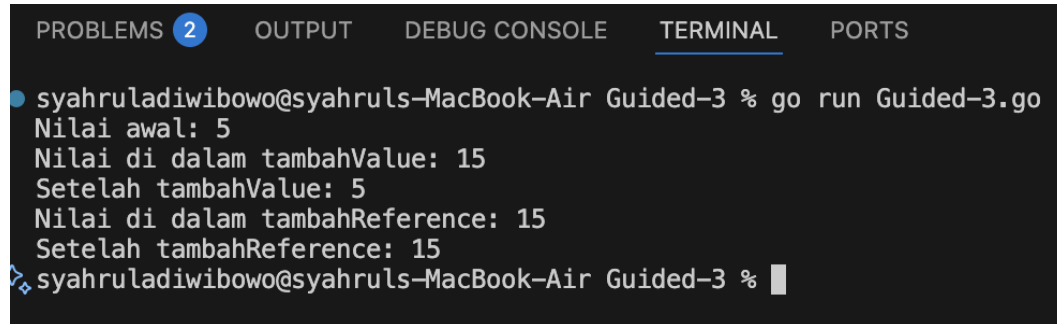
    for i := 0; i < n; i++ {
```

```
fmt.Scan(&data[i])
}

insertionSort(&data, n)

for i := 0; i < n; i++ {
    fmt.Print(data[i], " ")
}
}
```

Output :

A screenshot of a Go IDE's terminal window. The window has tabs for PROBLEMS (with a blue circle containing the number 2), OUTPUT, DEBUG CONSOLE, TERMINAL (which is selected and underlined), and PORTS. The terminal shows the command 'go run Guided-3.go' being executed. The output consists of five lines: 'Nilai awal: 5', 'Nilai di dalam tambahValue: 15', 'Setelah tambahValue: 5', 'Nilai di dalam tambahReference: 15', and 'Setelah tambahReference: 15'. The prompt 'syahruladiwibowo@syahruls-MacBook-Air Guided-3 %' is visible at the bottom of the terminal.

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
syahruladiwibowo@syahruls-MacBook-Air Guided-3 % go run Guided-3.go
Nilai awal: 5
Nilai di dalam tambahValue: 15
Setelah tambahValue: 5
Nilai di dalam tambahReference: 15
Setelah tambahReference: 15
syahruladiwibowo@syahruls-MacBook-Air Guided-3 %
```

Penjelasan:

Program tersebut menggunakan algoritma Insertion Sort untuk mengurutkan data integer secara descending atau dari nilai terbesar ke nilai terkecil. Data angka disimpan pada array bertipe `arrInt` dengan kapasitas maksimum yang ditentukan menggunakan konstanta `nMax`.

Pada function `insertionSort`, proses pengurutan dimulai dari elemen kedua array karena elemen pertama dianggap sudah terurut. Nilai elemen yang sedang diproses disimpan sementara ke dalam variabel `temp`, sedangkan variabel `j` digunakan untuk menelusuri posisi penyisipan yang tepat.

Selama proses perbandingan berlangsung, apabila nilai `temp` lebih besar dibanding elemen sebelumnya, maka elemen tersebut akan digeser ke kanan. Pergeseran dilakukan terus menerus hingga ditemukan posisi yang sesuai untuk menempatkan nilai `temp`.

Karena kondisi perbandingan menggunakan operator `>`, hasil akhir pengurutan akan menjadi descending atau urut dari besar ke kecil.

Pada bagian `main`, program terlebih dahulu membaca jumlah data yang akan dimasukkan, kemudian menerima input angka sebanyak jumlah tersebut.

Setelah function `sorting` dipanggil, seluruh data yang awalnya belum terurut akan ditampilkan dalam keadaan sudah terurut menurun.

C. Unguided

1. Soal Unguided 1

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Jawaban :

```
package main

import "fmt"

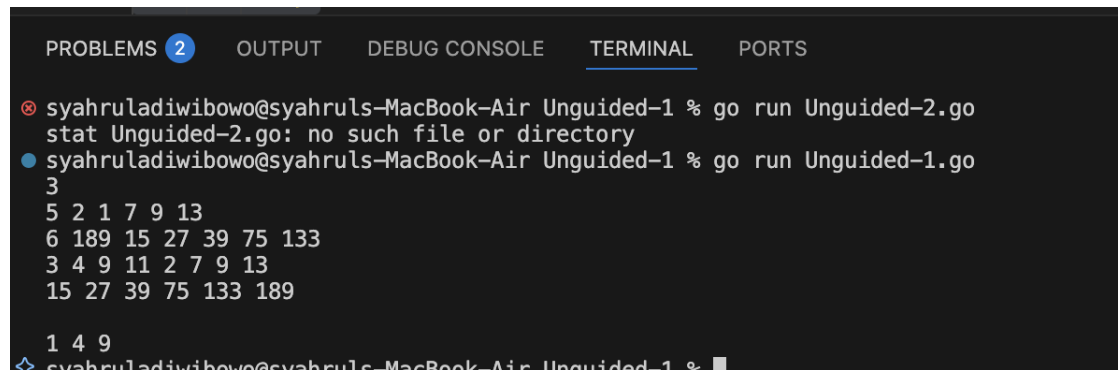
func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)
        arr := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }
        selectionSort(arr)
        for j, v := range arr {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(v)
        }
        fmt.Println()
    }
}
```

}

Output :

Penjelasan :



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
syahruladiwibowo@syahruls-MacBook-Air Unguided-1 % go run Unguided-2.go
stat Unguided-2.go: no such file or directory
syahruladiwibowo@syahruls-MacBook-Air Unguided-1 % go run Unguided-1.go
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 11 2 7 9 13
15 27 39 75 133 189

1 4 9
syahruladiwibowo@syahruls-MacBook-Air Unguided-1 %
```

Penjelasan:

Program ini digunakan untuk mengurutkan beberapa kumpulan data angka menggunakan algoritma Selection Sort secara ascending atau dari nilai terkecil ke terbesar.

Pada function selectionSort, data diterima dalam bentuk slice `[]int`. Proses sorting dilakukan dengan mencari nilai terkecil dari bagian array yang belum terurut. Variabel `minIdx` digunakan untuk menyimpan indeks elemen terkecil yang ditemukan selama proses perbandingan.

Loop pertama menentukan posisi data yang akan diisi, sedangkan loop kedua mencari nilai terkecil dari indeks setelahnya. Setelah nilai terkecil ditemukan, elemen pada posisi `i` akan ditukar dengan elemen pada posisi `minIdx`.

Pada function main, program membaca jumlah kasus sebanyak `n`. Untuk setiap kasus, program membaca jumlah data `m`, lalu membuat slice menggunakan `make([]int, m)`. Setelah semua data dimasukkan, function selectionSort dipanggil untuk mengurutkan data. Hasil akhirnya dicetak dalam satu baris untuk setiap kasus.

2. Soal Unguided 2

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main
```

```
import "fmt"
```

```
func selectionSortAsc(arr []int) {
```

```
    n := len(arr)
```

```
    for i := 0; i < n-1; i++ {
```

```
        minIdx := i
```

```
        for j := i + 1; j < n; j++ {
```

```
            if arr[j] < arr[minIdx] {
```

```
                minIdx = j
```

```
            }
```

```
        }
```

```
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
```

```
    }
```

```
}
```

```
func selectionSortDesc(arr []int) {
```

```
    n := len(arr)
```

```
    for i := 0; i < n-1; i++ {
```

```
        maxIdx := i
```

```
        for j := i + 1; j < n; j++ {
```

```
if arr[j] > arr[maxIdx] {  
    maxIdx = j  
}  
}  
arr[i], arr[maxIdx] = arr[maxIdx], arr[i]  
}  
}
```

```
func main() {  
    var n int  
    fmt.Scan(&n)  
    for i := 0; i < n; i++ {  
        var m int  
        fmt.Scan(&m)  
        arr := make([]int, m)  
        for j := 0; j < m; j++ {  
            fmt.Scan(&arr[j])  
        }  
        var ganjil, genap []int  
        for _, v := range arr {  
            if v%2 == 1 {  
                ganjil = append(ganjil, v)  
            } else {  
                genap = append(genap, v)  
            }  
        }  
    }  
}
```

```
selectionSortAsc(ganjil)
selectionSortDesc(genap)
for j, v := range ganjil {
    if j > 0 {
        fmt.Print(" ")
    }
    fmt.Print(v)
}
for j, v := range genap {
    if len(ganjil) > 0 || j > 0 {
        fmt.Print(" ")
    }
    fmt.Print(v)
}
fmt.Println()
}
}
```

Output :

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● syahruladiwibowo@syahruls-MacBook-Air Unguided-2 % go run Unguided-2.go
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 11 7 9 13 2
15 27 39 75 133 189

1 9 4
❖ syahruladiwibowo@syahruls-MacBook-Air Unguided-2 %
```

Penjelasan :

Program ini digunakan untuk memisahkan data bilangan ganjil dan genap, kemudian mengurutkannya dengan arah yang berbeda. Bilangan ganjil diurutkan secara ascending, sedangkan bilangan genap diurutkan secara descending. Function `selectionSortAsc` digunakan untuk mengurutkan data dari kecil ke besar. Cara kerjanya adalah mencari indeks nilai terkecil menggunakan variabel `minIdx`, lalu menukarnya ke posisi awal bagian yang belum terurut. Function `selectionSortDesc` digunakan untuk mengurutkan data dari besar ke kecil. Perbedaannya, function ini mencari nilai terbesar menggunakan variabel `maxIdx`, kemudian menukarnya ke posisi yang sesuai.

Pada function `main`, program membaca beberapa kasus uji. Setiap data yang dimasukkan akan dicek menggunakan operator modulus `%`. Jika `v%2 == 1`, maka data masuk ke slice ganjil. Jika tidak, data masuk ke slice genap. Setelah dipisahkan, data ganjil diurutkan menaik dan data genap diurutkan menurun. Kemudian hasilnya dicetak dengan urutan semua bilangan ganjil terlebih dahulu, lalu dilanjutkan dengan semua bilangan genap.

3. Soal Unguided 3

Soal:

3. Soal Unguided 3

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 17 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

jawaban :

```
package main
```

```
import "fmt"
```

```
func insertionSort(arr []int) {
```

```
    for i := 1; i < len(arr); i++ {
```

```
        key := arr[i]
```

```
        j := i - 1
```

```
        for j >= 0 && arr[j] > key {
```

```
            arr[j+1] = arr[j]
```

```
            j--
```

```
        }
```

```
        arr[j+1] = key
```

```
    }
```

```
}
```

```
func main() {
```

```
    var arr []int
```

```
    for {
```

```
        var x int
```

```
        fmt.Scan(&x)
```

```
        if x < 0 {
```

```
            break
```

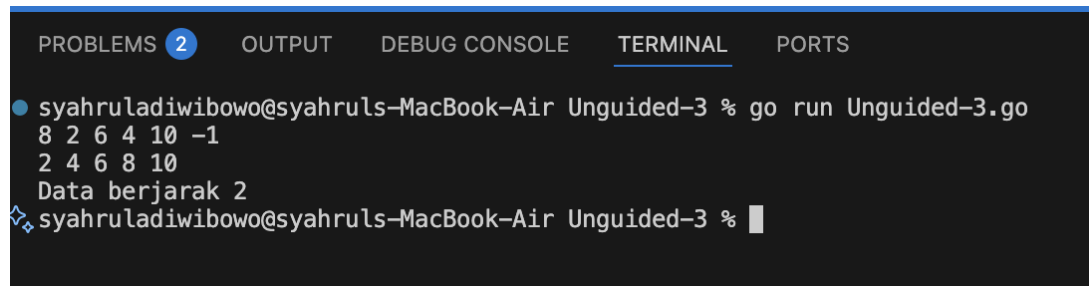
```
        }
```

```
arr = append(arr, x)
}
insertionSort(arr)
for i, v := range arr {
    if i > 0 {
        fmt.Print(" ")
    }
    fmt.Print(v)
}
fmt.Println()
if len(arr) < 2 {
    fmt.Println("Data berjarak tidak tetap")
    return
}
jarak := arr[1] - arr[0]
berjarak := true
for i := 2; i < len(arr); i++ {
    if arr[i]-arr[i-1] != jarak {
        berjarak = false
        break
    }
}
if berjarak {
    fmt.Printf("Data berjarak %d\n", jarak)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
```


}

}

Output :

A screenshot of a Go IDE's terminal window. The window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The prompt is 'syahruladiwibowo@syahruls-MacBook-Air Unguided-3 %'. The user has entered 'go run Unguided-3.go'. The program's output is displayed on the next three lines: '8 2 6 4 10 -1', '2 4 6 8 10', and 'Data berjarak 2'. The prompt is now 'syahruladiwibowo@syahruls-MacBook-Air Unguided-3 %' with a cursor at the end.

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
syahruladiwibowo@syahruls-MacBook-Air Unguided-3 % go run Unguided-3.go
8 2 6 4 10 -1
2 4 6 8 10
Data berjarak 2
syahruladiwibowo@syahruls-MacBook-Air Unguided-3 %
```

Penjelasan :

Program ini digunakan untuk membaca sekumpulan bilangan, mengurutkannya secara ascending menggunakan Insertion Sort, lalu memeriksa apakah jarak antar data setelah diurutkan bernilai tetap.

Pada function `insertionSort`, proses pengurutan dimulai dari elemen indeks ke-1. Nilai yang sedang diproses disimpan dalam variabel `key`. Kemudian elemen-elemen sebelumnya yang lebih besar dari `key` akan digeser ke kanan. Setelah posisi yang tepat ditemukan, `key` ditempatkan pada posisi tersebut.

Pada function `main`, input data dilakukan secara berulang menggunakan perulangan `for`. Program akan terus membaca angka sampai menemukan bilangan negatif. Bilangan negatif digunakan sebagai tanda berhenti dan tidak dimasukkan ke dalam slice.

Setelah data selesai dibaca, program mengurutkan data menggunakan `insertionSort`, lalu mencetak hasilnya. Setelah itu, program mengecek selisih antar elemen. Selisih pertama disimpan pada variabel `jarak`, lalu dibandingkan dengan selisih elemen-elemen berikutnya.

4. Soal Unguided 4

Soal:

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax ] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

jawaban :

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

type Buku struct {
    id string
    judul string
    penulis string
    penerbit string
    eksemplar int
    tahun int
    rating int
}

const nMax = 7919
var pustaka [nMax]Buku
var nPustaka int

func DaftarkanBuku() {
```

```
fmt.Print("")

scanner := bufio.NewScanner(os.Stdin)

scanner.Scan()

n, _ := strconv.Atoi(scanner.Text())

nPustaka = n

for i := 0; i < n; i++ {

    scanner.Scan()

    line := scanner.Text()

    data := strings.SplitN(line, " ", 7)

    pustaka[i].id = data[0]

    pustaka[i].judul = data[1]

    pustaka[i].penulis = data[2]

    pustaka[i].penerbit = data[3]

    pustaka[i].eksemplar, _ = strconv.Atoi(data[4])

    pustaka[i].tahun, _ = strconv.Atoi(data[5])

    pustaka[i].rating, _ = strconv.Atoi(data[6])

}

}

func CetakTerfavorit() {

    if nPustaka == 0 {

        fmt.Println("Tidak ada buku")

        return

    }

    maxIdx := 0

    for i := 1; i < nPustaka; i++ {
```

```
if pustaka[i].rating > pustaka[maxIdx].rating {  
    maxIdx = i  
}  
  
b := pustaka[maxIdx]  
  
fmt.Printf("%s %s %s %d\n", b.judul, b.penulis, b.penerbit, b.tahun)  
}
```

```
func UrutBuku() {  
    for i := 1; i < nPustaka; i++ {  
        key := pustaka[i]  
        j := i - 1  
        for j >= 0 && pustaka[j].rating < key.rating {  
            pustaka[j+1] = pustaka[j]  
            j--  
        }  
        pustaka[j+1] = key  
    }  
}
```

```
func Cetak5Terbaru() {  
    batas := 5  
    if nPustaka < 5 {  
        batas = nPustaka  
    }  
    for i := 0; i < batas; i++ {
```

```

b := pustaka[i]

fmt.Printf("%s %s %s %d\n", b.judul, b.penulis, b.penerbit, b.tahun)
}
}

func CariBuku() {
    scanner := bufio.NewScanner(os.Stdin)
    scanner.Scan()
    r, _ := strconv.Atoi(scanner.Text())
    low, high := 0, nPustaka-1
    found := false
    for low <= high {
        mid := (low + high) / 2
        if pustaka[mid].rating == r {
            b := pustaka[mid]
            fmt.Printf("%s %s %s %d\n", b.judul, b.penulis, b.penerbit, b.tahun)
            found = true
            break
        } else if pustaka[mid].rating < r {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }
    if !found {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

```

```
}
```

```
}
```

```
func main() {
```

```
    DaftarkanBuku()
```

```
    CetakTerfavorit()
```

```
    UrutBuku()
```

```
    Cetak5Terbaru()
```

```
    CariBuku()
```

```
}
```


Output :

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● syahruladiwibowo@syahruls-MacBook-Air Unguided-4 % go run Unguided-4.go
3
B01 Naruto Kishimoto Gramedia 10 2001 95
B02 OnePiece Oda Elex 7 1999 98
B03 Bleach Kubo MNC 5 2004 90
980OnePiece Oda Elex 1999
OnePiece Oda Elex 1999
Naruto Kishimoto Gramedia 2001
Bleach Kubo MNC 2004

OnePiece Oda Elex 1999
❖ syahruladiwibowo@syahruls-MacBook-Air Unguided-4 %
```

Penjelasan :

Program ini digunakan untuk mengelola data buku perpustakaan menggunakan struct, Insertion Sort, dan Binary Search. Data buku disimpan dalam struct Buku yang memiliki beberapa field, yaitu id, judul, penulis, penerbit, eksemplar, tahun, dan rating.

Array pustaka digunakan untuk menyimpan banyak data buku dengan kapasitas maksimum $nMax = 7919$. Variabel $nPustaka$ digunakan untuk menyimpan jumlah buku yang dimasukkan.

Function `DaftarkanBuku` digunakan untuk membaca jumlah buku dan seluruh data buku. Input dibaca menggunakan `bufio.Scanner`, kemudian setiap baris data dipisahkan menggunakan `strings.SplitN`. Data angka seperti eksemplar, tahun, dan rating dikonversi dari string ke integer menggunakan `strconv.Atoi`.

Function `CetakTerfavorit` digunakan untuk mencari buku dengan rating tertinggi.

Proses pencarian dilakukan secara linear dengan membandingkan rating setiap buku. Jika ditemukan rating yang lebih tinggi, indeks buku favorit akan diperbarui.

Function `UrutBuku` menggunakan algoritma Insertion Sort untuk mengurutkan buku berdasarkan rating secara descending. Buku dengan rating tertinggi akan berada di posisi awal array. Proses dilakukan dengan menyimpan data buku ke variabel `key`, lalu menggeser buku yang ratingnya lebih kecil ke kanan.

Function `Cetak5Terbaru` menampilkan maksimal lima buku pertama setelah proses pengurutan. Karena data sudah diurutkan berdasarkan rating tertinggi, maka lima data yang dicetak merupakan buku dengan rating paling tinggi.

Function `CariBuku` digunakan untuk mencari buku berdasarkan rating menggunakan Binary Search. Karena array sudah diurutkan secara descending, aturan pencariannya disesuaikan. Jika rating pada posisi tengah lebih kecil dari

rating yang dicari, pencarian dilanjutkan ke bagian kiri. Jika lebih besar, pencarian dilanjutkan ke bagian kanan.

D. Kesimpulan

Berdasarkan praktikum Modul 14 tentang Sorting, dapat disimpulkan bahwa proses pengurutan data merupakan salah satu konsep dasar yang sangat penting dalam pemrograman karena dapat mempermudah pengolahan, pencarian, dan pengelompokan data. Pada praktikum ini digunakan dua algoritma sorting dasar, yaitu Selection Sort dan Insertion Sort, yang masing-masing memiliki mekanisme kerja berbeda dalam menyusun data.

Selection Sort bekerja dengan cara memilih elemen terkecil atau terbesar dari bagian array yang belum terurut, kemudian menukarnya ke posisi yang sesuai. Algoritma ini memiliki alur yang sederhana dan mudah dipahami sehingga cocok digunakan untuk mempelajari dasar sorting. Sementara itu, Insertion Sort bekerja dengan cara menyisipkan elemen ke posisi yang tepat pada bagian data yang sudah terurut. Algoritma ini lebih efektif digunakan pada data yang berukuran kecil atau data yang hampir terurut.

Pada bagian Guided, algoritma sorting diterapkan pada berbagai tipe data seperti integer, string, dan struct mahasiswa. Pengurutan dilakukan baik secara ascending maupun descending sehingga memberikan pemahaman tentang penggunaan operator perbandingan dalam proses sorting.

Pada bagian Unguided, konsep sorting diterapkan pada kasus yang lebih kompleks seperti pengurutan bilangan ganjil dan genap dengan arah berbeda, pengecekan jarak antar data setelah diurutkan, hingga pengelolaan data buku perpustakaan menggunakan struct. Selain itu, digunakan juga algoritma Binary Search untuk melakukan pencarian data pada array yang sudah terurut sehingga proses pencarian menjadi lebih cepat dan efisien.

Melalui praktikum ini, dapat dipahami penggunaan array, slice, struct, function, pointer, serta kombinasi algoritma sorting dan searching dalam menyelesaikan berbagai permasalahan pemrograman. Praktikum ini juga memberikan pemahaman bahwa pemilihan algoritma harus disesuaikan dengan karakteristik data dan kebutuhan program agar proses pengolahan data dapat berjalan lebih optimal.

